Food Tinder

Aplicación de recomendación culinaria basada en compatibilidad molecular de alimentos

Nombre del alumno

11 de mayo de 2026

Resumen

El proyecto *Food Tinder* consiste en el diseño y desarrollo de una aplicación basada en recursos de Azure para almacenar, procesar y consumir datos alimentarios. La finalidad principal es recomendar combinaciones de alimentos que puedan funcionar bien dentro de una misma receta, utilizando como criterio principal la coincidencia de moléculas aromáticas entre ingredientes.

Índice

1. Introducción **3**

2. Objetivos del proyecto **3**

3. Esquema general de recursos y tecnologías **4**

3.1. Azure Storage Account y Data Lake 4
3.2. Azure Data Factory 4
3.3. Azure Databricks 4
3.4. Azure Cosmos DB 5
3.5. FastAPI 5
3.6. Azure Container Apps 5
3.7. Frontend 5

4. Transformación y procesamiento de datos **6**

4.1. Orquestación del pipeline 6
4.2. Importación de archivos en Databricks 6
4.3. Limpieza de datos 6
4.4. Normalización de nombres de alimentos 7

4.5. Unión de DataFrames . . . 7
4.6. Cálculo de compatibilidad molecular . . . 8
4.7. Ajuste por grupo alimentario . . . 8
4.8. Recomendaciones con varios ingredientes . . . 8
4.9. DataFrame de información nutricional . . . 9
4.10. Traducción y publicación de los datos . . . 9

5. Construcción de la API **10**
5.1. Endpoints principales . . . 10
5.2. Respuesta de la API . . . 10

6. Frontend y consumo de la aplicación **11**
6.1. Partes contruidas en el frontend . . . 11

7. Despliegue de la API **12**

8. Conclusiones **12**

Fuentes y datos utilizados **12**

1. Introducción

En este proyecto se lleva a cabo la construcción de una aplicación que utiliza distintos recursos de Azure para almacenar, transformar, procesar y consumir datos. La idea principal consiste en desarrollar un sistema capaz de indicar qué pares de alimentos pueden combinar bien dentro de una misma receta. Para ello, se comprueban sus coincidencias a nivel molecular y se establece un criterio adicional para descartar aquellas parejas que, aunque compartan muchas moléculas, no resulten adecuadas desde un punto de vista culinario por el grupo alimentario al que pertenecen.

Además de las recomendaciones de compatibilidad, los alimentos mostrados por pantalla se acompañan de información sobre sus macronutrientes. De esta forma, el usuario no solo obtiene posibles combinaciones de ingredientes, sino también un marco nutricional básico sobre cada alimento recomendado. Asimismo, estos alimentos también van acompañados de una lista de recetas generadas por *Google Gemini*

El funcionamiento general de la aplicación puede resumirse del siguiente modo: el usuario indica el alimento o alimentos que tiene disponibles para cocinar, la aplicación devuelve distintas ideas de combinaciones junto con los macros de cada alimento y presenta la opción de que el usuario seleccione una lista de alimentos de las recomendaciones proporcionadas para generar una lista de recetas.

El flujo de trabajo del proyecto se divide en dos grandes bloques. El primero está relacionado con la transformación y el procesamiento de los datos. El segundo se centra en la construcción, despliegue y consumo de la API, así como en la integración posterior con el frontend.

2. Objetivos del proyecto

Los objetivos principales del proyecto son los siguientes:

- Construir un pipeline de datos en Azure que permita organizar, transformar y procesar información alimentaria procedente de distintas tablas.
- Diseñar un sistema de recomendación de alimentos basado en la coincidencia de moléculas aromáticas incorporando un criterio adicional de compatibilidad culinaria basado en el grupo alimentario de cada ingrediente.
- Almacenar los resultados procesados en una base de datos NoSQL que facilite su consumo posterior desde una API.
- Desarrollar una API con FastAPI para consultar recomendaciones, alimentos disponibles en la base de datos, macronutrientes y recetas.

- Preparar la aplicación para su consumo desde un frontend y para su despliegue en la nube mediante Azure Container Apps.

3. Esquema general de recursos y tecnologías

La arquitectura del proyecto se organiza siguiendo un flujo de datos completo: ingesta, transformación, procesamiento, almacenamiento, exposición mediante API y consumo desde la interfaz de usuario. A continuación, se describen los principales recursos y tecnologías utilizados.

3.1. Azure Storage Account y Data Lake

Se utiliza una única *Storage Account* para centralizar el almacenamiento de los datos. Dentro de ella se separan los datos mediante contenedores que representan distintas capas del pipeline, como `raw` y `processed`. Esta separación permite organizar mejor el flujo de datos sin duplicar infraestructuras y reduciendo costes de almacenamiento.

Los datos originales se encuentran inicialmente en formato CSV. Antes de ser utilizados en las siguientes fases del pipeline, se transforman a formato Parquet, ya que este formato resulta más eficiente para el procesamiento distribuido y para la lectura desde entornos como Azure Databricks o Azure Data Factory. Esta conversión inicial se realiza en VSCode mediante la librería `pandas`.

3.2. Azure Data Factory

Azure Data Factory se emplea como herramienta de orquestación del flujo de datos. Mediante un *Data Flow*, se eliminan columnas que no son necesarias para el proyecto y se modifican ciertos tipos de datos. Posteriormente, los datos transformados se guardan en el contenedor correspondiente del Data Lake para ser consumidos por Databricks.

El uso de Data Factory permite organizar el proceso en actividades conectadas entre sí. De este modo, las transformaciones y el procesamiento pueden ejecutarse en un mismo *pipeline* en el orden indicado. Además, esta estructura facilita la detección de errores, ya que permite identificar en qué punto concreto de la acción se produce un fallo.

3.3. Azure Databricks

Azure Databricks se utiliza para el procesamiento principal de los datos. En esta fase se consumen los archivos Parquet almacenados en el Data Lake y se realizan las uniones necesarias entre las diferentes tablas disponibles.

El objetivo del procesamiento en Databricks es construir nuevas tablas ya preparadas para ser almacenadas en una base de datos. Estas tablas deben contener, por un lado, la

información necesaria para calcular y consultar la compatibilidad entre alimentos y, por otro lado, la información nutricional que será consumida por la API.

3.4. Azure Cosmos DB

Para el almacenamiento de los datos procesados se selecciona Azure Cosmos DB, una base de datos NoSQL. Esta elección se debe a que Cosmos DB permite almacenar documentos en formato JSON, lo que facilita representar estructuras complejas como listas de compatibilidades entre ingredientes.

El uso de una base de datos NoSQL resulta adecuado para este proyecto porque las recomendaciones no se almacenan como una tabla relacional clásica, sino como documentos que pueden contener alimentos, puntuaciones, listas de parejas compatibles y datos asociados. Además, Cosmos DB se integra de forma natural con APIs desarrolladas en Python, ya que tanto la API como la base de datos trabajan con estructuras similares a JSON, reduciendo la necesidad de transformaciones adicionales.

3.5. FastAPI

Una vez almacenados los datos procesados, se desarrolla una API utilizando el framework FastAPI. Esta API permite acceder a la información mediante peticiones HTTP y devolver los resultados en formato JSON.

FastAPI ofrece varias ventajas para este proyecto. En primer lugar, permite definir endpoints de forma clara y sencilla. En segundo lugar, valida automáticamente los datos de entrada. Por último, genera documentación interactiva que facilita probar y revisar el funcionamiento de los endpoints durante el desarrollo.

3.6. Azure Container Apps

Azure Container Apps se utiliza para desplegar la API en la nube y permitir su acceso mediante un endpoint público. De este modo, la aplicación no depende del entorno local ni de la ejecución manual en un equipo concreto.

Este servicio facilita el despliegue de aplicaciones a partir de contenedores y evita tener que gestionar directamente infraestructura como servidores, redes o escalado. Así, la API puede ser utilizada por otros usuarios de forma sencilla, manteniendo un funcionamiento consistente y un modelo de coste basado en el uso.

3.7. Frontend

El frontend será la parte de la aplicación encargada de permitir la interacción del usuario con el sistema. Desde la interfaz, el usuario podrá introducir uno o varios alimentos, consultar combinaciones recomendadas, ver los macronutrientes asociados y solicitar

recetas generadas a partir de los ingredientes seleccionados. Todo esto se llevará a cabo mediante la framework *Vue*, con el fin de aprender a desarrollar un frontal con las herramientas requeridas en mi posible puesto dentro de la empresa.

4. Transformación y procesamiento de datos

4.1. Orquestación del pipeline

El pipeline se construye en Azure Data Factory. En él se añade un Data Flow encargado de eliminar las columnas que no se necesitan para el proyecto, conservando aquellas que permiten realizar posteriormente las uniones entre tablas. Esta actividad se conecta con un notebook de Azure Databricks, donde se lleva a cabo el procesamiento principal.

Con esta estructura se consigue tener un orquestador principal que controla el flujo completo. Primero se ejecutan las transformaciones iniciales y después el procesamiento en Databricks. Esta organización facilita tanto la automatización como la revisión de errores.

4.2. Importación de archivos en Databricks

Una vez realizada la transformación inicial, los archivos Parquet se importan en Databricks desde el contenedor correspondiente del Data Lake. A partir de estos archivos se construyen los DataFrames que serán utilizados en las distintas fases del procesamiento.

Las DataFrames principales utilizados en el proyecto son los siguientes:

- El de alimentos, que contiene información general sobre cada ingrediente.
- El de contenidos, que relaciona alimentos con nutrientes y compuestos.
- El de compuestos, que contiene información sobre moléculas.
- El de sabores, que permite identificar características aromáticas asociadas a las moléculas.
- El de nutrientes, utilizada para obtener información nutricional.

4.3. Limpieza de datos

En la limpieza de datos no se eliminan todos los valores nulos, sino únicamente aquellos que afectan directamente al funcionamiento del modelo. En concreto, se eliminan registros con valores nulos en identificadores o nombres, ya que estos campos son necesarios para realizar correctamente los joins entre DataFrames.

Por el contrario, los nulos en variables descriptivas se mantienen cuando no afectan al análisis. En el DataFrame de contenidos, los nulos relacionados con nutrientes se eliminan

cuando estos son necesarios para el análisis nutricional. En cambio, en el caso de los compuestos, algunos valores nulos se mantienen, ya que no siempre es obligatorio disponer de todos los datos asociados a una molécula.

4.4. Normalización de nombres de alimentos

En el DataFrame de alimentos se realiza una normalización del nombre de los ingredientes para garantizar una búsqueda más robusta desde la API. Esta normalización se aplica sobre el dataset principal de alimentos, ya que el input introducido por el usuario se comparará con el nombre de los ingredientes.

El proceso de normalización incluye los siguientes pasos:

1. Convertir todos los nombres de alimentos a minúsculas.
2. Eliminar espacios al inicio y al final de cada cadena de texto.
3. Eliminar tildes y caracteres diacríticos mediante la librería `unicodedata`.
4. Eliminar signos de puntuación mediante expresiones regulares.
5. Singularizar los nombres de los alimentos mediante la librería `inflect`.
6. Encapsular los pasos anteriores en una función de normalización.
7. Aplicar dicha función al DataFrame mediante una UDF.
8. Crear una nueva columna con el nombre normalizado de cada alimento.

De este modo, se reducen los errores de búsqueda producidos por diferencias de mayúsculas, acentos, signos de puntuación o formas plurales.

4.5. Unión de DataFrames

El proyecto utiliza un DataFrame de sabores y otro que relaciona los identificadores de sabores con los identificadores de moléculas. Estos dos se unen para identificar las moléculas asociadas a características aromáticas.

Una vez identificadas las moléculas aromáticas, se realiza la unión entre el DataFrame de alimentos, el de contenidos y el de de moléculas y sabores. El objetivo es obtener, para cada alimento, el conjunto de moléculas aromáticas asociadas.

A partir de este último DataFrame se seleccionan las columnas correspondientes al alimento y a las moléculas. Después, las filas se agrupan por alimento para contar el número total de moléculas asociadas a cada uno.

4.6. Cálculo de compatibilidad molecular

Para comparar alimentos a nivel molecular se construye un nuevo DataFrame con pares de ingredientes. Para ello, se realiza un *self join* sobre el DataFrame de alimentos y moléculas, generando todas las combinaciones posibles de alimentos. En este proceso se evita conservar duplicados equivalentes, como `food1-food2` y `food2-food1`.

La compatibilidad molecular se calcula mediante una puntuación normalizada que tiene en cuenta tanto el número de moléculas compartidas como el número total de moléculas de cada alimento. No se considera únicamente el número absoluto de moléculas compartidas, ya que esto podría favorecer artificialmente a los alimentos con un mayor número de compuestos registrados.

Una forma natural de expresar este tipo de normalización es mediante una medida basada en la proporción entre moléculas compartidas y moléculas totales consideradas:

$$\text{score}_{mol}(A, B) = \frac{|M_A \cap M_B|}{|M_A \cup M_B|}, \tag{1}$$

donde M_A y M_B representan los conjuntos de moléculas asociadas a los alimentos A y B, respectivamente. En la versión final del documento, esta expresión debe ajustarse exactamente a la fórmula implementada en el notebook de Databricks.

Tras calcular la puntuación de compatibilidad, se conservan únicamente aquellas parejas que superan el umbral definido en el proyecto.

4.7. Ajuste por grupo alimentario

Para mejorar la precisión culinaria de las recomendaciones, se incorpora un criterio adicional basado en el grupo de alimento al que pertenece cada ingrediente. La idea es favorecer combinaciones que, además de compartir moléculas aromáticas, tengan sentido desde un punto de vista culinario.

Para ello, se define un DataFrame de compatibilidades entre grupos alimentarios. Este permite aplicar bonificaciones o penalizaciones según la pareja de grupos a la que pertenezcan los alimentos comparados. Así, dos alimentos con una alta coincidencia molecular pueden recibir una puntuación menor si pertenecen a grupos que no suelen combinar bien en una receta.

A partir de la puntuación molecular y del ajuste por grupo alimentario se obtiene una puntuación final. Posteriormente, se crea un ranking de combinaciones por alimento y se seleccionan las mejores recomendaciones para cada ingrediente.

4.8. Recomendaciones con varios ingredientes

Para recomendaciones con varios ingredientes no se almacenan combinaciones triples o cuádruples, ya que este enfoque escalaría muy mal al aumentar el número de alimentos.

En su lugar, se almacenan compatibilidades por pares, agrupadas por alimento.

Cuando el usuario introduce varios ingredientes, la API combina en tiempo real las recomendaciones asociadas a cada uno de ellos. Para ello, calcula la intersección entre los resultados disponibles y genera una puntuación agregada que permite ordenar las recomendaciones finales.

Este planteamiento reduce el volumen de datos almacenado y mantiene la flexibilidad de la aplicación, ya que permite trabajar con múltiples ingredientes sin precomputar todas las combinaciones posibles.

4.9. DataFrame de información nutricional

Además del DataFrame de compatibilidades, se crea otro específico para el endpoint de información nutricional. Este contiene una columna con el nombre del ingrediente y varias columnas adicionales, cada una correspondiente a un macronutriente.

Para construirla, se unen el DataFrame de alimentos y el de nutrientes a través del de contenidos. Después, se filtran los nutrientes de interés: energía, proteínas, grasas y carbohidratos. También se priorizan los alimentos en crudo cuando esta información está disponible.

Como el DataFrame de contenidos puede dividir un mismo alimento en diferentes partes o tipos de preparación, se agrupa por alimento, macronutriente y tipo de preparación. Finalmente, se calcula una media del contenido estándar de cada macronutriente para obtener una representación única por alimento.

4.10. Traducción y publicación de los datos

Una vez generadas los DatFrames finales necesarios para la aplicación, se realiza una fase previa a la publicación en la base de datos.

Dado que las fuentes originales contienen los nombres de los alimentos en inglés, se incorpora un proceso de traducción mediante la API de Google Translator. Esta traducción se aplica sobre los resultados finales del procesamiento, antes de su publicación en Cosmos DB. De esta manera, se conserva la estructura generada durante el procesamiento y únicamente se añade una capa lingüística que permite mostrar la información en español dentro de la aplicación. La traducción se aplica principalmente a los nombres de los alimentos y a aquellos campos textuales que deben ser visibles para el usuario.

Finalmente, los DataFrames se cargan en Cosmos DB, que actúa como base de datos de consulta para el backend de la aplicación.

5. Construcción de la API

La API se construye con FastAPI y actúa como capa intermedia entre la base de datos y el frontend. Su función principal es recibir las peticiones del usuario, consultar Cosmos DB y devolver los resultados en formato JSON.

5.1. Endpoints principales

Los endpoints previstos para la API son los siguientes:

- Endpoint para obtener los macronutrientes de un alimento.
- Endpoint para consultar la compatibilidad entre los alimentos que aporte el usuario.
- Endpoint para buscar alimentos por nombre, consiguiendo así consultar los alimentos disponibles en la base de datos.
- Endpoint para pedir recetas a la API de Google Gemini a través de un prompt que contiene los alimentos dados por el usuario y las recomendaciones seleccionadas por este.

5.2. Respuesta de la API

Las respuestas de la API se devuelven en formato JSON. Este formato resulta adecuado porque puede ser consumido directamente por el frontend y porque coincide con la estructura documental utilizada en Cosmos DB.

En el caso de las recomendaciones, la respuesta incluye la lista de ingredientes compatibles y la puntuación de compatibilidad. En el caso de los macronutrientes, la respuesta contiene los valores nutricionales asociados al alimento solicitado.

En el caso de las recetas, la respuesta contiene una lista de recetas generadas a partir de los ingredientes introducidos por el usuario y de las recomendaciones seleccionadas. Para ello, la API envía estos alimentos a un modelo generativo mediante un prompt específico, solicitando recetas breves, plausibles y estructuradas. La respuesta generada se procesa posteriormente para separar cada receta en campos independientes, como el título y el cuerpo de la receta, lo que facilita su representación en el frontend.

Además, se incorpora un endpoint auxiliar para consultar los distintos tipos de alimentos disponibles en la base de datos. Este endpoint permite buscar coincidencias relacionadas con un alimento introducido por el usuario y devolver variantes existentes en la base de datos. Por ejemplo, si el usuario introduce un término general, la API puede devolver distintos alimentos cuyo nombre contiene dicho término o que pertenecen a una variante concreta. Esta funcionalidad mejora la interacción con la aplicación, ya que ayuda al usuario a seleccionar ingredientes realmente disponibles en la base de datos.

6. Frontend y consumo de la aplicación

El frontend será el punto de entrada para el usuario final. Desde la interfaz, el usuario podrá escribir uno o varios alimentos, consultar sus mejores combinaciones, revisar los macronutrientes de cada ingrediente y solicitar recetas basadas en las recomendaciones obtenidas.

La comunicación entre el frontend y la API se realizará mediante peticiones HTTP. El frontend enviará los alimentos introducidos por el usuario a los endpoints correspondientes y mostrará la respuesta recibida de forma ordenada.

6.1. Partes contruidas en el frontend

El frontend se ha desarrollado con Vue y se ha organizado en componentes y vistas diferenciadas para separar la lógica de navegación, búsqueda y visualización de resultados. La aplicación incluye una barra de navegación común a todas las rutas y varias vistas principales, entre ellas la pantalla de inicio, la vista de búsqueda de combinaciones y la vista de consulta de macronutrientes.

La vista de búsqueda constituye el núcleo del frontend. En ella, el usuario puede introducir uno o varios alimentos mediante una barra de búsqueda enlazada al estado interno del componente con `v-model`. Conforme se escribe, el frontend consulta la API para mostrar sugerencias de alimentos disponibles. Una vez añadidos, estos alimentos se almacenan en una lista reactiva y pueden enviarse al endpoint de recomendaciones.

Tras la consulta, el frontend muestra una lista de alimentos recomendados. Cada recomendación incorpora dos acciones: consultar macronutrientes o añadir el alimento a la lista de ingredientes seleccionados para generar recetas. Esta segunda lista también se representa dinámicamente en pantalla y permite eliminar elementos antes de realizar la petición al endpoint de recetas.

La consulta de macronutrientes y la visualización detallada de recetas se han implementado mediante ventanas superpuestas sobre la interfaz principal. Esta solución permite mantener al usuario en la misma vista mientras accede a información complementaria. En el caso de las recetas, primero se presenta una lista de recetas generadas y posteriormente se muestra el contenido completo de la receta seleccionada en una ventana modal de mayor tamaño.

Finalmente, se han incorporado estados de carga, mensajes de error y mecanismos para reiniciar la búsqueda, con el fin de mejorar la robustez y la experiencia de uso del sistema.

7. Despliegue de la API

Para que la API pueda ser probada y utilizada sin depender del entorno local, se plantea su despliegue en Azure Container Apps. Para ello, la API se empaqueta en un contenedor y se publica en la nube. Una vez desplegada, Azure Container Apps proporciona un endpoint público desde el que se pueden realizar peticiones.

Este despliegue facilita la validación del proyecto por parte de otros usuarios, ya que no es necesario ejecutar manualmente la API en el equipo del desarrollador. Además, permite mantener un comportamiento consistente entre entornos y delegar en Azure la gestión de la infraestructura necesaria.

8. Conclusiones

El proyecto *Food Tinder* integra distintas tecnologías de Azure para construir una aplicación completa basada en datos. El sistema parte de información alimentaria en bruto, la transforma y procesa mediante un pipeline, almacena los resultados en Cosmos DB y los expone mediante una API desarrollada con FastAPI.

La principal aportación del proyecto es el uso de la coincidencia molecular como criterio para recomendar combinaciones de alimentos. No obstante, este criterio se complementa con un ajuste por grupo alimentario, ya que una alta similitud molecular no garantiza por sí sola que dos alimentos funcionen bien dentro de una receta.

La estructura planteada permite ampliar el proyecto en fases posteriores, incorporando nuevas funcionalidades como búsqueda bilingüe, generación automática de recetas, construcción de una imagen Docker para un posterior despliegue sobre un recurso de Azure, recomendaciones con varios ingredientes y una interfaz web completa para el usuario final.

Fuentes y datos utilizados

La fuente principal de datos alimentarios utilizada en el proyecto procede de FoodDB. Su esquema puede consultarse en:

https://foodb.ca/schema